OWL University of Applied Sciences and Arts

Faculty of Computer Science

Department of Environmental Engineering and Applied Computer Science

Implementation of AI Agents Using Large Language Models for Agentic Workflows

Rahaman Walter Dongmepi Wendji

rahaman.dongmepi@stud.th-owl.de

15481090

7.Semester

2025-2026

Submitted in fulfilment of the requirements for the research project of owl university of applied sciences and arts

DECLARATION OF OWN WORK

Declaration & Consent:

This submission is my own work. Any quotation from, or description of the work of others is acknowledged herein by reference to the sources, whether published or unpublished.

Signature: Rahaman Walter Dongmepi Wendji

Abstract

In recent years, the rapid advancement of Large Language Models (LLMs) has enabled the development of sophisticated Artificial Intelligence (AI) agents capable of automating complex workflows, thereby saving time and resources. However, improper configuration of these agents can lead to inefficiencies and unintended outcomes.

This research paper explores key agent architectures and design patterns essential for building effective agentic workflows. It presents a comparative analysis of commercial LLMs (such as GPT, Gemini, and Claude) and open-source models (such as Qwen and LLaMA) to highlight the advantages and limitations of each approach.

Furthermore, the study examines two primary implementation methods: using low/no-code platforms (e.g., n8n) and manual development through agent frameworks such as LangChain and other Agent Development Kits (ADKs). In the second part, a multi-agent workflow is implemented using both commercial and local LLMs with the LangChain framework, connected locally to a Model Context Protocol (MCP) server for interaction with Gmail and Calendar services. The source code for the implementation is available at my GitHub Repository. This research provides foundational insights into the design and implementation of agentic workflows powered by LLMs.

Keywords: Large Language Model, AI Agent, MCP, agentic workflow, agent framework, agent architecture, multi-agent architecture, design pattern

Contents

1 Introduction

Over the last decades, Artificial Intelligence (AI) has been applied in several systems and in many forms. We can notice a rapid evolution of AI from specialized systems designed for narrow tasks to increasingly sophisticated architectures capable of autonomous operation across diverse domains (13). Years after years, the need for an AI system that can interact with their environment, make decisions, and achieve complex goals has grown significantly. That is where AI Agent comes in.

In contrast to traditional AI systems that execute predetermined algorithms, AI agents have the capability, based on environmental feedback and accumulated experiences, to perceive, reason, and act autonomously. The differentiation between AI agents and other AI systems lies mainly in their architecture and operational capabilities. Common AI systems usually operate within predefined parameters, whereas Agentic systems or AI agents are becoming more capable of acting independently to achieve goals without needing constant human direction. Several companies implement those agentic systems in workflows and processes to achieve better results and performance. This brings us to the question of which methods are best suited to implement an AI Agent on a rapid evolving word? What should be considered when developing these agentic systems?

This research project aims to develop and evaluate an autonomous AI agent, powered by a local LLM and then by cloud-based AI service like ChatGPT, Gemini, Claude, that automates workflows and executes tasks with minimal or no human intervention. We begin by reviewing the literature on agent systems, defining what agent systems are and how they are built. We then present different implementation approaches, tools and technologies that are necessary to build an AI agent. After that we implement AI agent using local LLM and then AI cloud services. Last but not least, we analyse and evaluate the outcomes and dicuss about different approaches.

2 Literature Review

2.1 AI Agents and Autonomous Task Execution

In contrast to zero-shot prompting of a large language model where user send queries into a boundless text field and obtain a result without additional input, agents allow for more complex interaction and orchestration. Specifically, agentic systems have a notion of planning, loops, reflection, and other control structures that considerably leverage the model's inherent reasoning capabilities to tackle a task end-to-end. Combine with the use of tools, plugins, and function calling, agents gain the ability to perform more general-purpose tasks (21).

2.1.1 Definition and Core Concepts

Agent-based AI is distinguished from traditional AI by its need to generate dynamic behaviors that are founded on an understanding of environmental contexts (5). So an AI agent is an autonomous system that, based on the received data, makes rational decisions and acts within its environment to achieve specific goals (3). An AI agent possesses a "reasoning engine". The agent leverages this engine to make rational decisions based on the environment and its actions (3).

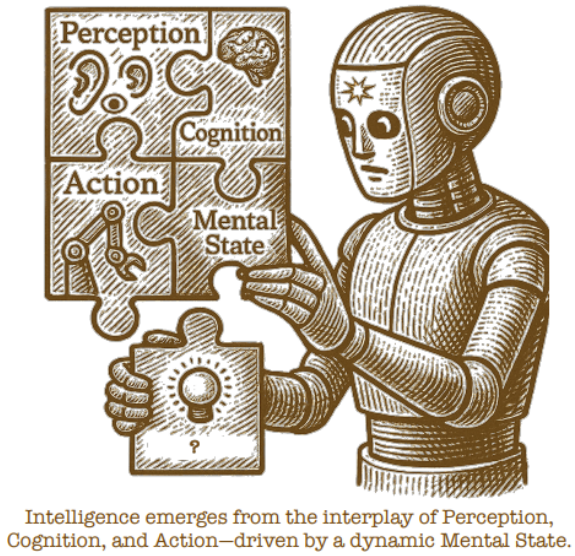


Figure 1: Core concepts of AI agents, (19)

What characteristics transform a model into a truly intelligent agent? The difference resides not in scale but in structure - in the presence of a mind that can see, feel, remember, plan, and act. A mind that perceives their world, carry memory through time, adapts to change, and chases goals beyond the next token (19). In Figure 1 we can notice the core components of an intelligent agent, Perception, Cognition,

Action, and Mental State. These core faculties form a living loop: perception feeds cognition, cognition selects action, and action reshapes perception (19). The cognition encompasses a range of other faculties such as memory, world model, emotion, goals, and learning. It is the center of control that organizes thought, plans, and decisions. Then comes perception, the gateway from the world to the mind, and action, the channel through which the mind shapes the world. Each module is inspired by the brain's capacity to solve problems. This part builds the foundation for everything. The components listed above are the classical core components of an AI agents. These map directly the four fundamental elements of modern AI agents, such as large language models (as known as LLM), contextual memory, external functions and tools, and routing and orchestration. This correspondence provides clarity on how traditional cognitive architectures translate into contemporary agent implementations.

Cognition represents the agent's reasoning, planning, and decision-making capabilities. In modern AI agents, this cognitive function is primarily driven by **Large Language Models(LLMs**). These LLMS are the central reasoning engine, providing:

- Natural language understanding and generation.
- Logical reasoning through frameworks like ReAct, Chain-of-Thought
- Planning capabilities vie iterative reasoning loops.
- Problem decomposition and multi-step task execution(25).

Mental State englobes the agent's internal representations, beliefs, and world model. This matches the **contextual memory** systems of modern agents. (24). Contextual memory includes:

- Short-term memory: current conversation context and working information
- Long-term memory. Persistent knowledge across interactions
- Belief states: Internal representations of the world derived from sensory inputs.
- World models: compressed spatial and temporal representations of the environment (29).

The agent's ability to execute tasks and interact with the environment represents **action**. This corresponds to **external functions** and tools in modern architectures.

Gathering, processing and interpreting sensory input from an environment to build awareness belongs to **perception**. This function represents the **routing and orchestration** mechanisms in modern agents. This provides input processing, attention mechanisms, context filtering, and multi-model integration.

2.1.2 Agent Architectures and Design Patterns

Agent architecture is the blueprint for designing software agents and intelligent systems, outlining components and control flows that allow agents to act autonomously, plan, and interact with environment or other agents. In contrast to traditional pre-defined and hard-coded workflows, AI Agents can determine the positive and negative aspects of decisions and select the best alternative among several options for executing a task (28). The real power of AI Agents comes from the interaction of these elements:

- Input processing: Agents obtain information from various sources - direct user questions, system events, or data from external sources (3).
- Decision-making: In opposite to simple chatbots, AI agents use multi-step prompting techniques to make decisions. Using chains of specialized prompts enables agents to tackle complex tasks beyond the reach of single-shot responses(3).
- Action execution: Modern LLMs like Claude, GPT, Llama produce structured outputs that serve as function calls to external systems (3).
- Learning and adaption: Through various mechanisms, some agents are able to improve over time, from simple feedback loops to sophisticated model updates(3).

Architectures shape how agents process tasks, reason, and coordinate their actions. It is a misconception to assume that the most important decision in building an AI product or system is choosing between GPT-5, Claude, Gemini or an open-source model. With a solid architecture, even the most powerful AI model will drain resources, waste GPU time, create technical debt, and underperform compared to a simpler model built on a strong foundation (31).
We differentiate between the **single-agent architecture** and the **multi-agent architecture**. These are the core architectural models. But before choosing an architecture, we should primary determine the end-to-end workflow the system requires. In other words, what is the system fundamentally required to do from the moment it receives input until it produces output? Is the workflow short and linear, or long with several phases? Can some phases run in parallel, or are there strict dependencies between stages? Does the flow involve multiple roles, services, or data types? Does the process require coordination, retries, or adaptability? The system's workflow determines the most suitable architecture, whether single-agent or distributed multi agent.

Single Agent Architectures

These architectures leverage a single language model to manage all reasoning (evaluating input and deciding what to do), planning(breaking goals into steps), tool execution (calling functions or APIs) autonomously, and interacting with tools (using capabilities beyond the model, like databases or search). The agent uses a system prompt and any tools required to complete their task (21). The 2 show a simple single architecture design.

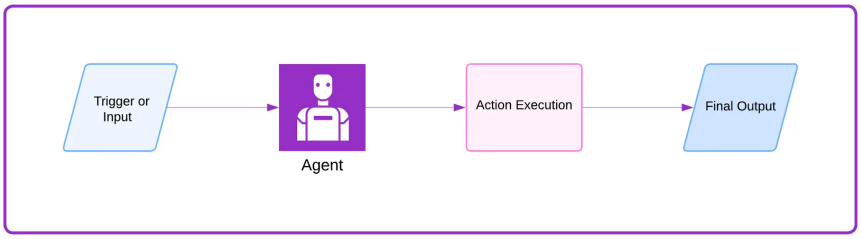


Figure 2: single agent architecture, (31)

The use of single a single agent architecture have many benefits like:

- Simplicity: It's simple to design, implement, test, and maintain.
- Lower overhead: There's no need for inter-agent communication, so computational and networking overhead is lower.
- More predictable behavior: Randomness and unforeseen interactions are reduced.
- Transparent debugging: It is not difficult to trace and diagnose issues with a single entity managing all logic (6).

However, some challenges are present when we decide to use single agent architectures:

- Lack of scalability: Greater complexity leads to reduced single-agent performance.
- Limited modularity: Separating concerns and extending functionality incrementally becomes more difficult.
- Reduced robustness: malfunction of an agent may compromise the stability of the entire system.
- Limited suitability for distributed problems: Single-agent systems are not well-suited for tasks that are inherently distributed across multiple dynamic entities.

We distinguish many patterns that are used in single-agent architectures:

- **The single-agent pattern**: That is the simplest form of a single-agent architecture. The agent system reacts to a trigger, processes a task, and returns an output. Memory, planning, or external interaction is not involved. Only **input**, **reasoning**, and **response** are required. See the figure 3 An example

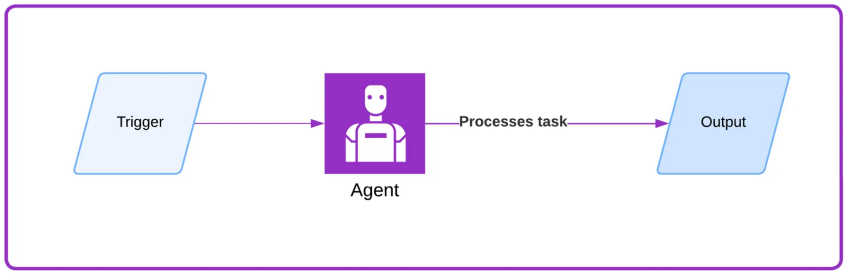


Figure 3: Single-agent pattern (31)

of this is the open-source automation agent "bumpgen" that monitors projects for new package releases and makes pull requests to update dependencies—handled entirely by one agent that observes, fetches, and updates, with no need for multi-agent coordination (36).

- **The memory-augmented agent pattern**: When the system requires access to a past context such as previous user interactions, historical data, or external states to make a better decision, it is useful and recommended to use this pattern Figure 4. It allows agents to respond with awareness of past events.

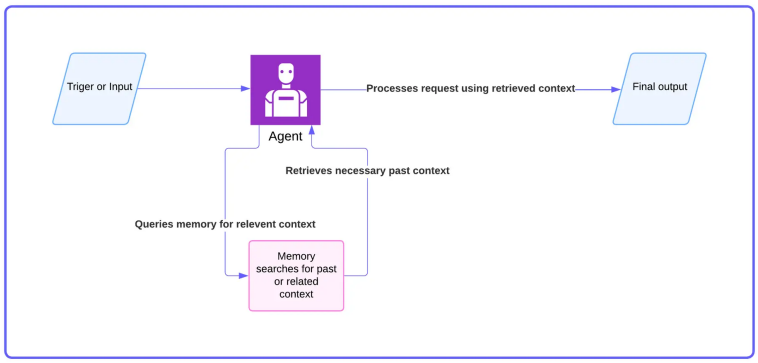


Figure 4: Single agent memory augmented (31)

A typical example are conversational copilots that remember user preferences (2).

- **The tool-using agent pattern**: This pattern refers to an AI agent architecture in which the capabilities of agents are extended using external tools such as APIs, software functions, or web services to perform tasks that extend beyond its internal capabilities (26). Typically, these agents must determine when to use its own capabilities and when to delegate sub-tasks to an external tool. For efficient API interaction, an MCP (Model Context Protocol) layer is generally introduced (see figure 5). MCP is a standardized framework designed to enable seamless interactions between AI systems (such as LLM) and external tools, resources, or data sources(11). It standardizes how requests

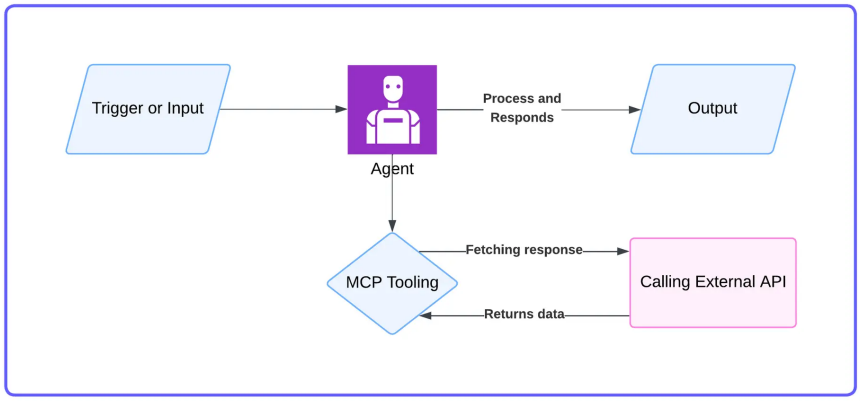


Figure 5: Single agent tool using architecture, (31)

and responses are structured.

- **The planning-agent pattern**: this is a design pattern for AI agents in which the agent creates an explicit plan, a high-level roadmap of steps or sub-tasks, before it begins to execute actions. The agent then performs each action in order, dynamically adapting based on understanding dependencies between tasks and execution tracking (see figure 6).

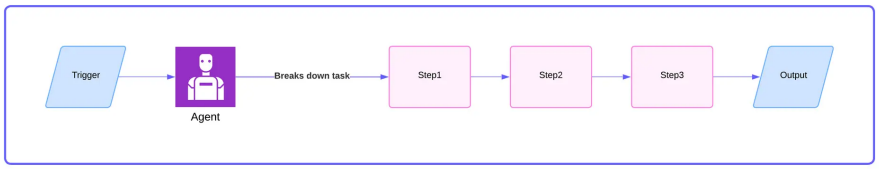


Figure 6: Planning-agent pattern, (31)

- **The reflection-agent pattern**: This is an AI design framework in which an agent evaluates its own reasoning or outputs, learns from those evaluations, and iteratively improves its performance without external retraining, eliminating the need for human intervention. This pattern is valuable for systems that learn from previous outcomes to enhance future performance.

Multi-agent architectures

A multi-agent architecture refers to a system design composed of multiple autonomous agents, each possessing specialized capabilities, knowledge, or role that interact cooperatively or competitively to tackle complex problems beyond the capacity of any single agent(33).

Multi-agent architectures are widely used in distributed AI, robotics, simulations, and systems that require autonomous or semi-autonomous modules. In these architectures, agents are designed with autonomy and the ability to perceive, act and communicate so they can interact effectively within a shared system. A multi-agent

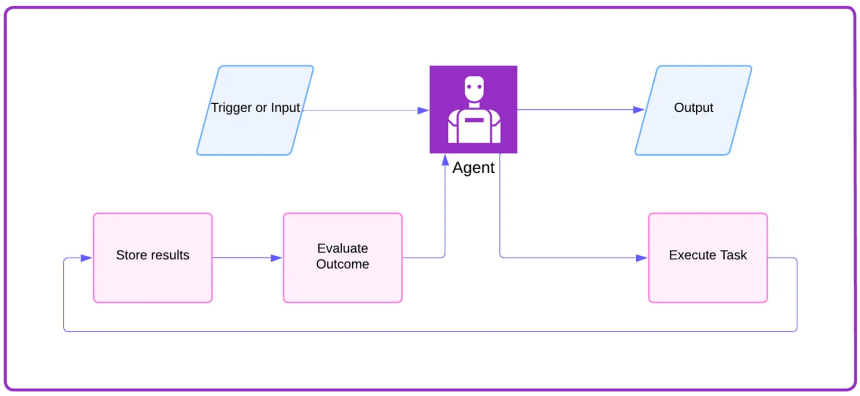


Figure 7: Single-agent reflection architecture, (31)

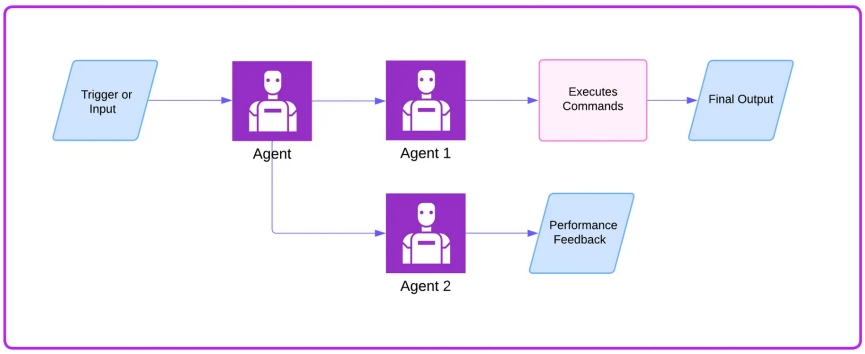


Figure 8: Multi-agent architecture, (31)

architecture is recognized by these key features:

- Each agent acts autonomously, but collaborates or coordinates with other agents with the system.
- Agents have specific knowledge or skills, such as analysis, planning, data retrieval, or communication.
- Tasks are usually decomposed, delegated, and orchestrated by a central manager agent or distributed among peers according to the architecture style.

The use of multi-agent architectures has several benefits such as:

- Modularity: It simplifies the development, testing, and maintenance of systems (30).
- Specialization: By focusing on their specific domains of expertise, agents contribute to improve system performance(30).
- Scalability: Large and complex tasks can be addressed by scaling agent numbers and diversity(30).
- Redundancy: Individual agent failures do not affect the system, making it more robust (6).
- Parallelism: Multiple agents execute different subtasks at the same time for better performance(6).

- Emergent behavior: Complex behaviors emerge from agent interactions, enabling solutions to problems difficult for single agents(6).

It can become challenging to implement multi-agent architecture. We can face the following problems:

- Increased complexity: Requires sophisticated coordination, communication, and conflict resolution(6).
- Higher resource consumption: Implementing multiple agents can lead to greater consumption of computing and communication resources(6).
- Nondeterministic outcomes: Emergent behaviors and agent interaction can make system behavior more difficult to predict and test(6).
- Risk of compounding errors: Accumulated errors in reasoning, planning, and goal evaluation can lead to system failure, necessitating corrective mechanisms(6) (3)
- Debugging difficulty: Due to distributed responsibilities, it is harder to trace bugs and understand failures(6).
- Optimization difficulty: Because of its complexity, optimizing the system requires greater effort (6).

To ensure the right use of multi-agent systems, it is recommended to consider the following guidelines:

- Define the agent's tasks and outline how they determine if their objectives are met.
- The orchestrator must clearly define the distribution of work and specify the scope of each subtask to avoid redundant efforts and overlooked subtasks.
- Define the complexity of tasks and subtasks, and allocate agents appropriately. (6)

The multi-agent architecture can be implemented using different pattern, depending on the use case.

- The **Supervisor Agent Pattern** is a coordination and control pattern in which one or more higher-lever agents(called supervisors or manager agents) oversee, coordinate, delegate tasks to lower-level worker or specialist agents. A practical application of this pattern is the Tippy's multi-agent system for drug discovery laboratory automation (7) (see fig 9).

  The system employs a supervisor-agent paradigm in which five specialized agents leverage domain-specific tools accessed through the Model Context Protocol (MCP).
- The **hierarchical pattern**: This is a structural design principle in which agents are organized across multiple levels of abstraction or authority. Each layer manages different aspects of decision-making, coordination, and learning(20).

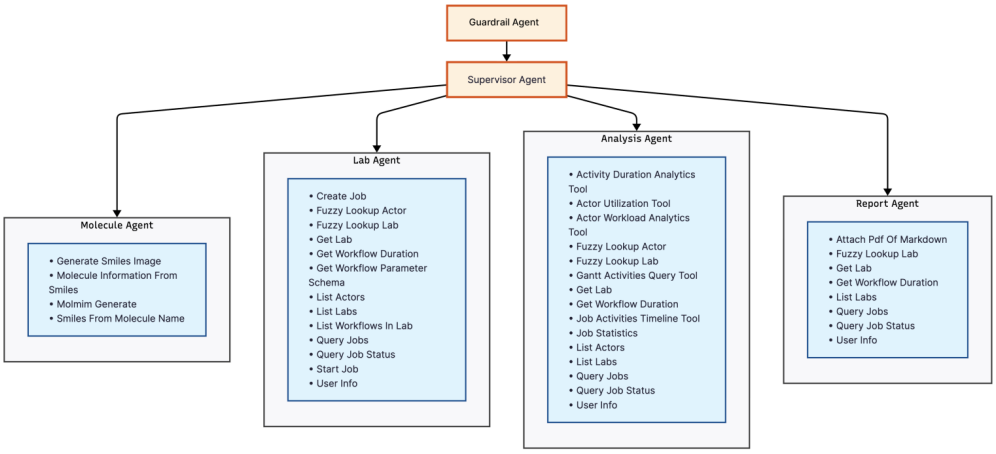


Figure 9: Multi-agent architecture showing the Supervisor Agent coordinating with four specialized agents - Molecule, Lab, Analysis, and Reports agents - along with the Safety Guardrail Agent for laboratory safety validation (7).

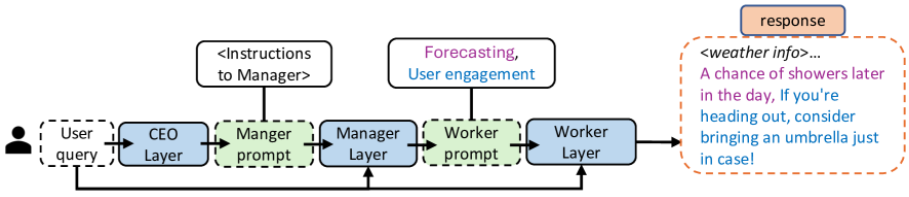


Figure 10: Hierarchical multi-agent workflow(HMAW) (20)

The hierarchy resembles organizational structures in human systems Figure 10. The layers are structured in three main level:

- **High-Level(CEO or Chief Agents)**: Handle long-term or global objectives. They perform intent interpretation, task decomposition, and goal prioritization(20).
- **Mid-Level (Coordinator or Manager Agents)**: Acting as intermediaries between planning and execution, they continuously monitor task progress and adapt workflows in response to changing conditions(20).
- **Low-Level (Worker or Skill Agents)**: They execute domain-specific actions, learn localized policies, and report outcomes to higher-level components(20).

The following figure 11 demonstrates how how this architecture handles user query

- The **competitive pattern**: in this pattern, multiple agents concurrently tackle the same problem, independently proposing alternative solutions. A dedicated evaluator agent assesses all submissions and selects the most appropriate solution according to predefined criteria, including speed, accuracy, creativity, and cost-efficiency.

  This approach is advantageous when diversity of reasoning or redundancy of

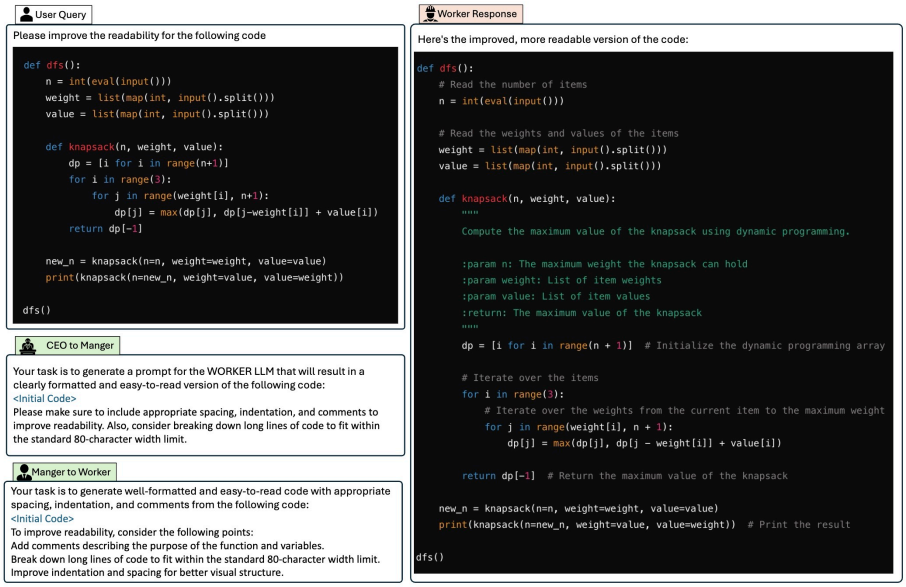


Figure 11: A case study of HMAW on the CodeNet Dataset (20)

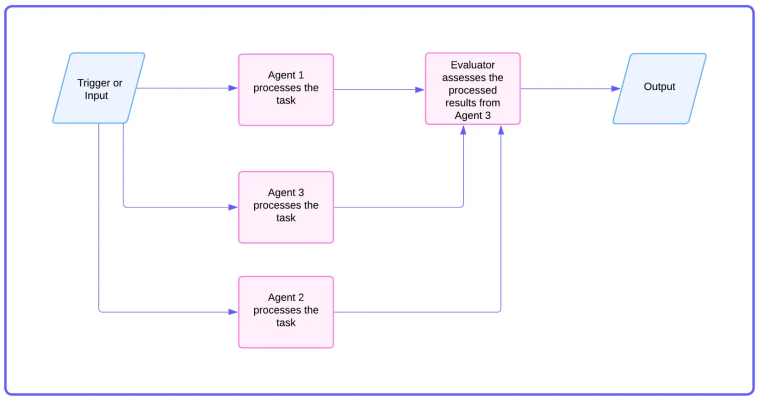


Figure 12: Multi agent competitive architecture, (31)

solutions contribute to better outcomes.

- The **parallel pattern**: Also known as a concurrent pattern, multiple specialized subagents perform a task or sub-tasks independently at the same time. The outputs of the subagents are combined to produce the final consolidated response. In the parallel pattern, a parallel workflow agent autonomously manages the scheduling and execution of subagents, eliminating the need for an AI model to orchestrate their interactions (9). The following diagram shows a high-level view of a multi-agent parallel pattern:
- The **swarm pattern**: in this pattern, multiple specialized agents work together to iteratively refine a solution to a complex problem. The swarm pattern employs a dispatcher agent to direct user requests to a collaborative ensemble of specialized agents. The dispatcher agent analyzes the incoming request and determines which agent within the swarm is best suited to initiate the task(see figure 14). In this pattern, each agent can interact with each

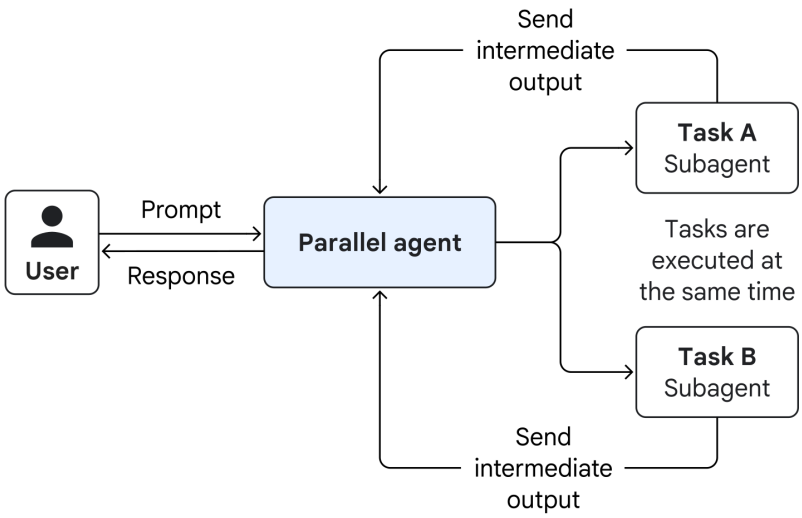


Figure 13: Architecture of the multi-agent parallel design pattern (9)

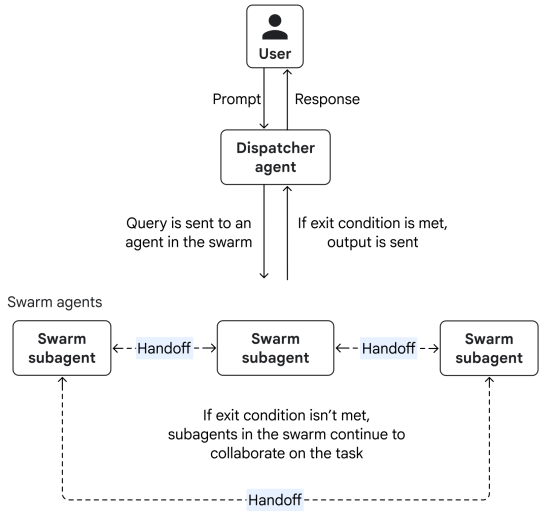


Figure 14: Architecture of the multi-agent swarm design pattern (9)

other agent, allowing them to share findings, critique proposals, and build on each other's work to iteratively improve a solution(9). Within the swarm, any agent can delegate the task to another agent identified as more capable of handling the next phase, or transmit the finalized output to the user via the coordinator agent.

2.2 Large Language Models for Agentic Workflows

Large Language models(LLMs) are increasingly being used to drive agentic workflows, systems in which models autonomously reason, plan, and execute multi-step tasks. Agentic workflows outline how LLMs or multi-agent systems progress through reasoning, decision-making, tool use, and execution phases to achieve human-like autonomy (22).Rather than relying on static prompts, they leverage dynamic orches-

tration frameworks that manage exceptions, plan tasks, and adapt their structure to pursue diverse goals. The choice of LLM used in an agentic workflow can significantly affect the results. To achieve optimal performance for a given use case, it is important to distinguish between commercial LLM APIs (e.g., GPT-5, Claude 4.5, Gemini 2.5) and locally deployed or self-hosted models(e.g., Llama, Deepseek, Qwen). Some research in 2025 emphasizes that while cloud APIs optimize for performance and convenience, local deployments excel in sovereignty, reproducibility, and pipeline flexibility(12).

2.2.1 Commercial API Services (ChatGPT, Claude, Gemini)

Commercial AI API such as GPT(OpenAI), Claude(Anthropic), and Gemini(Google) are leading a shift toward agentic workflows, where autonomous systems can think, plan, and take action across several steps to tackle complex tasks. Each ecosystem offers distinct advantages and integrations to design agentic systems.

GPT APIs(OpenAI)

GPT-5, released in August 2025, introduces native agentic orchestration through "structured tool use," auto-function chaining, and multimodal grounding throughout text, vision, and audio (23).

- In recent testing, the system demonstrates notable robustness against adversarial prompt injections, preventing about 85% of attacks(27).
- The model enables autonomous multi-step reasoning through controllable API tool nodes for retrieval-augmented generation (RAG), code execution, and external API interactions.
- With the tightly integration with OpenAI Realtime API and the Functions API, agent workflows call external services or persist task state(27).
- For AI agents, this enables deterministic control loops, which is ideal for research automation, data analyis assistants, and interarctive software agents (23).

Claude 4.5(Anthropic)

Claude 4.5 builds on Claude 3 Opus by introducing long-term memory, secure tool integration, and constitutional multi-persona reasoning (37).

- Claude 4.5 highlights alignment and latency: Its constitutional AI principles are embedded directly in the model weights rather than applied through external filtering (1).
- Anthropic Workbench and the Claude API enable users to chain reasoning steps with JSON-schema-controlled tool calls, recommended for planning

agents that must stay interpretable.

- Claude stands out at multi-document synthesis and goal directed dialogue, useful for research agents and legal-reasoning tasks.

Gemini 2.5 (Google Deepmind)

Gemini 2.5 pro expands DeepMind's Gemini 1.5 pro with large-context multimodality and tight integration into Google's infrastructure(Google Workspace).

- The system is designed to connect its reasoning to live, multimodal data sources (such as the web, video, and apps) in real time, which is a core requirement for autonomous cross-modal agents (10).
- Through Vertex AI, Gemini's API offers structured function-calling and external tool integration capabilities similar to those of the OpenAI ecosystem, while natively connecting with App Script, Google Drive, and other Workspace services (32).
- Gemini 2.5 pro is well-suited for enterprise agent workflows that require access to business data or Google ecosystem resources, with a focus on privacy, low latency, and multi-agent orchestration across cloud endpoints (32).

API Provider	**Agentic Framework**	**Key Strengths**	**Deployment Ecosystem**
GPT (OpenAI)	AgentKit, Custom GPTs	Tool calling, API coordination, structured workflows	Azure, GitHub, PromptLayer
Claude (Anthropic)	Claude Agent SDK, Claude Code, Agentic Flow	Context management, modular sub-agents, cost routing	AWS Bedrock, Vertex AI
Gemini (Google)	ADK, Vertex AI, LangGraph	Multimodal reasoning, real-time search grounding, cloud orchestration	Google Cloud, Vertex AI

Table 1: Comparative Overview of Commercial Agentic API Providers

2.2.2 Local LLM Solutions

Local large language models are increasingly employed to drive agentic workflows, where agents autonomously reason, plan, and execute multi-step processes. The

use of local LLMs rather than cloud APIs has distinct implications for privacy, architecture, trade-offs, and operational challenges.

Core Benefits of Local LLMs in Agentic Workflows

- **Privacy and Data Control**: Running LLMs locally ensures sensitive data remain in the local infrastructure, which is crucial for domains with strict compliance or confidentiality requirements. This provides a major benefit compared to cloud-hosted APIs, which can introduce risks related to data leakage or regulatory non-compliance (12).
- **Cost Efficiency**: Running open-source models(such as Qwen3, DeepSeek V3.2, Gemma 3, Llama 4) through platforms like Ollama, LM Studio, GPT4All, or LocalAI reduce usage-based cloud fees and give the control over scaling.
- **Customization and Tuning**: local LLMs can be fine-tuned or adapted for specific workflows or domains, achieving higher performance in specialized or confidential tasks that commercial APIs may not (14).
- **Latency & Scalability**: Local models offer low-latency for single users but may require careful orchestration for multi-user or parallel agentic tasks. Using tools like n8n and Docker help manage workflow execution.
- **Reduced external dependency**: The Use of local models eliminates reliance on third-party uptime, rate limits, and unexpected API changes, providing long-term operational autonomy(12).

Setting up agentic workflows with local LLMs requires high engineering effort, including model deployment, prompt optimization, scheduling, context management, and multi-agent orchestration. Local deployment demands managing inference hardware(CPUs/GPUs), especially for multi-stage, latency sensitive tasks. Therefore, running advanced local LLMs for multi-agent workflows requires adequate GPUs and efficient resource schedulers to maintain acceptable throughput and latency.

Comparison of Local and Cloud/API-based LLMs

Aspect	**Local LLMs**	**Cloud/API LLMs**
Privacy	Full data control	Data exposure to provider
Cost	Upfront hardware, lower run costs	Pay-by-request, possibly higher Total Cost of Ownership
Fluency/Capabilities	May lag latest commercial models	Usually higher fluency, up-to-date
Reliability	Needs robust local system design	Relies on API uptime/API changes
Customization	Can finetune for task/domain	Limited or not permitted

Table 2: Comparison of Local LLMs vs. Cloud/API LLMs (12)

3 Implementation Approaches

In this section, two different approaches will be explored for implementing a multi-agent system following the supervisor pattern. In the first approach, a code-based implementation is employed using the LangChain and LangGraph frameworks, which are well-suited for rapid prototyping of agentic workflows. This approach tests the integration of both a cloud-based large language model (such as Gemini or GPT-5) and a locally hosted model (such as Qwen 3.5). The second approach utilizes n8n, a low- or no-code platform that enables the development of agentic workflows with minimal programming knowledge, offering a faster and more accessible alternative to manual implementation.

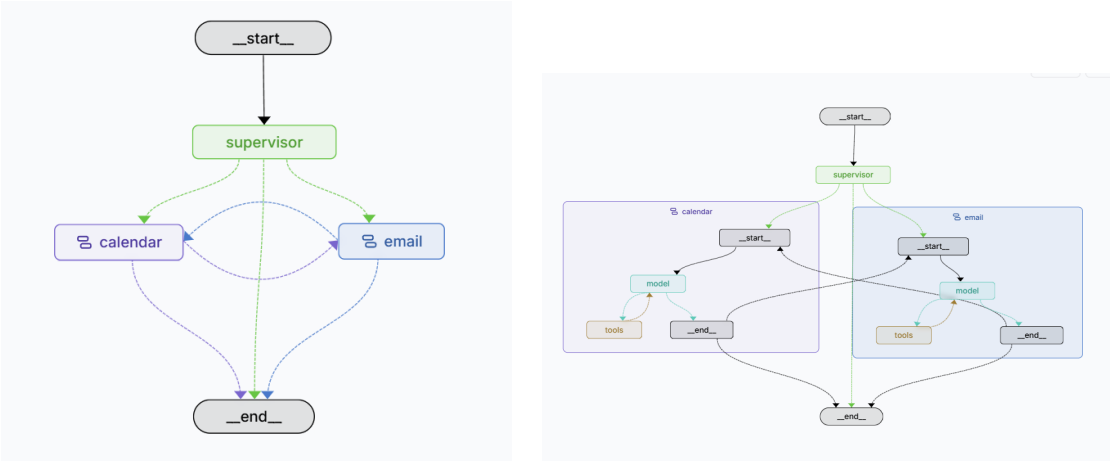
The goal of the project is to develop a fully featured chat agent by combining multiple specialized agents within a single unified interface. To achieve this, a multi-agent architecture following a supervisor pattern will be implemented. In this setup, two specialized agents collaborate seamlessly, including a calendar agent whose task is solely to check for availability in set events or appointments, then an email agent that will send email to a specific person. Users interact with a single interface. Behind the scenes, the orchestrator or supervisor intelligently routes requests to the right specialist.

3.1 Code-Based Approach

To build this chat agent, the LangChain and LangGraph frameworks are employed, as they provide comprehensive documentation and strong community support for developing agentic workflows. LangChain enables rapid prototyping, and its extensive integration ecosystem and beginner-friendly API make it well-suited for users who are new to LLM-based frameworks. LangGraph, on the other hand, offers a robust toolset for constructing complex multi-agent systems. Its core functionality lies in providing stateful abstractions, along with advanced features such as time-travel debugging, human-in-the-loop interrupts, and strong fault-tolerance mechanisms.

For debugging and observability, LangSmith is integrated into the system. Since the workflow requires scheduling events and sending email messages, the implementation uses the Google MCP server (35), which supplies the necessary tools and services for agent interaction. The system is accessed and managed through the LangChain/LangGraph Studio, which provides a user-friendly interface for communicating with agents and viewing their responses.

One benefit of **LangGraph Studio** is the ability to **visualize the agent architecture** and how each module interacts with others. Figure 15a represents the



(a) Architecture of the multi-agent system *(b) Detailed architecture of the multi-agent system*

Figure 15: Multi-agent Architecture with the supervisor pattern in langGraph Studio

multi-agent structure, which is directly derived from the code implementation. Figure 15b illustrates the interaction of each agent with the tools. The structure cannot be modified manually as it automatically updates according to changes in the implementation logic.

The figure illustrates three agents: a **supervisor agent** and two subagents—the **calendar agent** and the **email agent**. The supervisor agent acts as a **router**, delegating incoming requests to the appropriate subagent. The calendar agent handles event scheduling, while the email agent manages drafting and sending emails. If a request involves both scheduling and emailing, the tasks are executed sequentially. Once a subagent completes its task, it passes the remaining task to the next subagent.

Whether all agents use the same Large Language Model (LLM) or different ones depends on the specific task and the expected output. LangChain provides a toolset that facilitates the integration of various LLMs.

```
#model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
model = ChatOpenAI(model="gpt-5-mini", temperature=0)
#model = ChatOllama(model="qwen3:8b", temperature=0)
```

Figure 16: LLM definition (4)

The code snippet above defines the LLM used by the agents. It is straightforward to switch from one model to another. The LangChain documentation (LangChain) offers detailed guidance on integrating any LLM or external tool.

```python
calendar_agent = create_agent(
    model,
    tools=calendar_mcp_tools,
    system_prompt=CALENDAR_AGENT_PROMPT,
)

email_agent = create_agent(
    model,
    tools=gmail_mcp_tools,
    system_prompt=EMAIL_AGENT_PROMPT,
)
```

Figure 17: Initialization of calendar and email subagents (4)

Figure 17 shows the initialization of the calendar and email subagents. Each subagent includes the model, the tools it can access, and the prompt that defines its guardrails and capabilities.

```python
graph.set_entry_point("supervisor")

graph.add_conditional_edges(
    "supervisor",
    router,
    {
        "calendar": "calendar",
        "email": "email",
        "END": END,
    }
)
```

(a) Routing logic of the supervisor agent

```python
graph.add_conditional_edges(
    'calendar',
    router,
    {
        "email": "email",
        "END": END,
    }
)
graph.add_conditional_edges(
    'email',
    router,
    {
        "calendar": "calendar",
        "END": END,
    }
)
```

(b) Routing logic of subagents

Figure 18: Routing logic of the multi-agent system (4)

Figure 18 illustrates the request routing process. Based on the content of a request, the supervisor either delegates it to the appropriate subagent or discards it if it does not involve scheduling or emailing (see Figure 18a). Once a subagent completes its assigned task, the supervisor may forward any remaining task to another subagent or terminate the process if all actions have been completed (see Figure 18b). The routing logic is implemented in the `router` method.

3.2 Low-Code/No-Code Approach

In this approach, the n8n platform is utilized because it provides a versatile workflow automation environment that seamlessly combines artificial intelligence (AI) with business process automation. It offers the flexibility of custom coding along with the speed and simplicity of a no-code interface. The platform features extensive documentation, numerous tutorials on YouTube, and a variety of pre-built templates that make it easy to create AI agents quickly. In addition, n8n includes built-in tools for monitoring and debugging workflows. This makes it an ideal choice for individuals with little or no programming experience who wish to develop AI agents rapidly

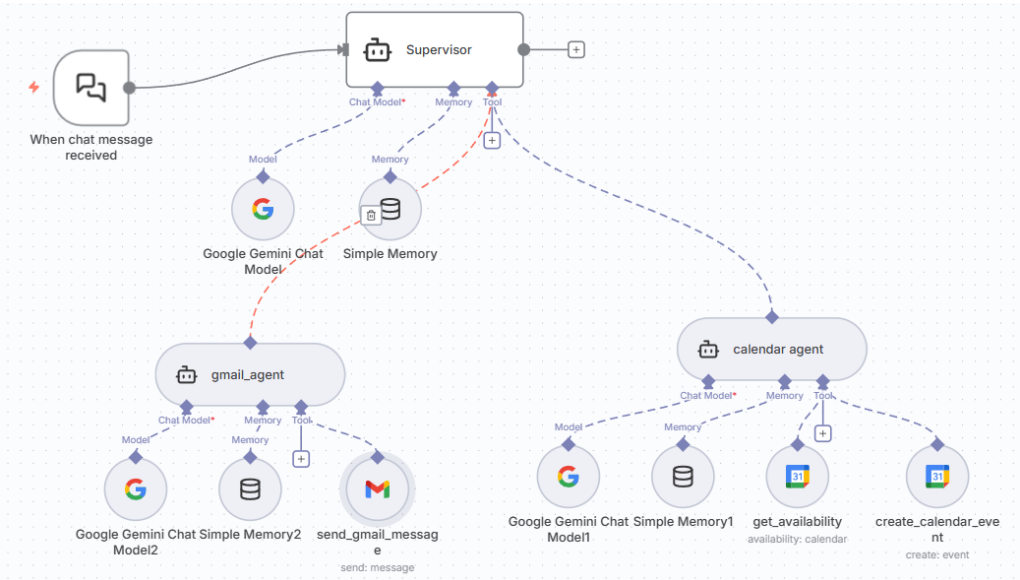


Figure 19: Implementation of a multi-agent system using n8n

The main advantage of n8n lies in its ease of use. The setup of core components and tools is accomplished through an intuitive drag-and-drop interface. Figure 19 presents a visual representation of the agentic workflow, which helps illustrate the overall architecture and communication between components. Four main elements can be identified in the figure: a chat component and three AI agent components. The primary agent acts as the orchestrator, while the other two agents function as subagents responsible for specific tasks. The calendar agent checks availability and schedules events, while the email agent handles message composition and delivery.

The workflow is initiated when a user sends a message through the chat component. Upon receiving this message, the orchestrator agent determines which subagent is best suited to handle the task and routes the request accordingly. The supervisor's primary responsibility is to direct each request to the appropriate agent who can perform the required operation.

This coordination relies heavily on an effective prompt design. Without a well-designed prompt or system message, the orchestrator may fail to assign tasks correctly. Therefore, defining clear and precise prompts is essential, as they establish both the scope and the operational boundaries of each agent. The following example illustrates the prompt used by the supervisor.

```
You are a helpful personal assistant coordinator.
Your role is to acknowledge user requests and provide
    helpful context.
For scheduling/calendar requests, explicitly mention '
    calendar' or 'scheduling' in your response to route to
    the calendar_agent.
For email requests, explicitly mention 'email' or 'mail' in
     your response to route to the email_agent.
For general questions that don't require calendar or email
    actions, mention 'general question' or 'information' in
    your response.
Keep your responses brief and friendly.
Always clearly indicate which agent should handle the
    request by mentioning the relevant keywords (calendar,
    email, or general).
```

3.3 Results and Evaluation

Building an effective agentic workflow requires a thorough analysis and understanding of the intended use cases. With this insight, selecting the appropriate architecture and design pattern becomes significantly easier. Another key consideration is the method of implementation. This research paper explores two main approaches: the code-based approach and the low- or no-code approach. Each presents distinct advantages and limitations.

The **code-based approach** offers extensive customization, scalability, enhanced security, comprehensive testing capabilities, and full control over the system. However, it also introduces higher development complexity, greater maintenance demands, a steeper learning curve, and challenges in managing context effectively. An additional factor to evaluate is the choice between using a cloud-based LLM or an open-source model.

By contrast, the **low- or no-code approach** enables rapid development and fosters easier collaboration, especially for non-developers. It allows for faster deployment, lower technical barriers, cost efficiency, and seamless integration with other tools.

However, the reliance on predefined modules often limits flexibility in implementing complex logic. Moreover, teams depend on the platform's capabilities, update cycles, and inherent limitations, which can constrain workflow evolution as requirements grow more sophisticated. For the

While **cloud-based LLMs** offer excellent scalability and performance, they pose significant concerns regarding data privacy and control. **Open-source models**, on the other hand, provide greater customization, transparency, cost management, and data security. Nonetheless, they demand substantial computational resources, as these models must be self-hosted or run locally.

4 Conclusion

This research provided a comprehensive overview of various agent architectures and design patterns, offering guidance for selecting the most appropriate approach for specific use cases. The findings highlight that choosing the right architecture and pattern depends heavily on the goals and complexity of the task at hand. The study also examined the trade-offs between cloud-based and open-source large language models (LLMs). While open-source models enhance data privacy and allow greater customization, they require substantial computational resources to achieve strong performance. Conversely, cloud-based LLMs deliver lower latency and better scalability but can become costly due to high API usage fees.

A balanced strategy is therefore recommended: local LLMs are well-suited for rapid prototyping and testing, whereas cloud-based models can be integrated later to achieve higher accuracy and performance once the solution is validated. In some cases, a hybrid setup may provide the best results—using local models for simpler tasks and cloud LLMs for more complex operations.

In the final part of this study, a simple multi-agent system was implemented using two different methods: the code-based approach and the low- or no-code approach. The results show that the code-based method offers greater flexibility, control, and customization potential, but requires significant programming expertise and presents a steep learning curve. In contrast, the low- or no-code approach is ideal for users with limited technical experience, enabling faster prototyping and a clearer visual understanding of the workflow's logic and interactions. However, this method involves dependency on the platform's reliability and API costs, which can limit control and scalability in production environments.

It is essential to note that Agents are not independent applications but are always integrated into an existing system.

Acknowledgements

This work was supported in part by my supervisor Prof. Dr. Ralf Hesse.

References

[1] Anthropic (2025). Introducing claude sonnet 4.5. Retrieved October 29, 2025, from https://www.anthropic.com/news/claude-sonnet-4-5.

[2] AWS (2025). Memory-augmented agents - aws prescriptive guidance. Retrieved October 25, 2025, from https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/memory-augmented-agents.html.

[3] Dmitrievna, Y. (2025). Ai agents explained: From theory to practical deployment. Retrieved October 15, 2025, from https://blog.n8n.io/ai-agents/.

[4] Dongmepi Wendji, R. W. (2025). Multi-Agent Personal Assistant. Retrieved ., from https://github.com/WalterWendji/multi-agent

[5] Durante, Z., Sarkar, B., Gong, R., Taori, R., Noda, Y., Tang, P., Adeli, E., Lakshmikanth, S. K., Schulman, K., Milstein, A., Terzopoulos, D., Famoti, A., Kuno, N., Llorens, A., Vo, H., Ikeuchi, K., Fei-Fei, L., Gao, J., Wake, N., and Huang, Q. (2024). An Interactive Agent Foundation Model. Retrieved from https://arxiv.org/abs/2402.05929v2.

[6] Edupont (2025). Single-agent and multi-agent architectures - dynamics 365. Microsoft Learn. Retrieved October 23, 2025, from https://learn.microsoft.com/en-us/dynamics365/guidance/resources/contact-center-multi-agent-architecture-design.

[7] Fehlis, Y., Crain, C., Jensen, A., Watson, M., Juhasz, J., Mandel, P., Liu, B., Mahon, S., Wilson, D., Lynch-Jonely, N., Leedom, B., Fuller, D., and Artificial, Inc. (2025). Technical implementation of tippy: Multi-agent architecture and system design for drug discovery laboratory automation. *Artificial, Inc.*, pages 1–2. Retrieved ., from https://arxiv.org/pdf/2507.17852

[Google] Google. Agent development kit. Retrieved October 29, 2025, from https://google.github.io/adk-docs/.

[9] Google (2025). Choose a design pattern for your agentic ai system. Retrieved October 24, 2025, from https://cloud.google.com/architecture/choose-design-pattern-agentic-ai-system#single-agent-system.

[10] Google (2025). Gemini 2.5 pro. Retrieved October 27, 2025, from https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-pro.

[11] Hou, X., Zhao, Y., Wang, S., and Wang, H. (2025). Model context protocol (mcp): Landscape, security threats, and future research directions. Retrieved ., from https://arxiv.org/abs/2503.23278

[12] Koraag, T., Wagner, N., Dobslaw, F., and Gren, L. (2025). Comparing open-source and commercial llms for domain-specific analysis and reporting: Software engineering challenges and design trade-offs. Retrieved ., from https://arxiv.org/abs/2509.24344

[13] Krishnan, N. (2025). AI Agents: Evolution, Architecture, and Real-World Applications. Retrieved ., from https://arxiv.org/abs/2503.12687v1

[14] Lan, X., Feng, J., Lei, J., Shi, X., and Li, Y. (2025). Localgpt: Benchmarking and advancing large language models for local life services in meituan. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, 2:12–12. Retrieved ., from https://arxiv.org/pdf/2506.02720

[LangChain] LangChain. Build a supervisor agent – docs by langchain. Retrieved ., from https://docs.langchain.com/oss/python/langchain/supervisor#openai

[LangChain] LangChain. Integration packages - Docs by LangChain. Retrieved November 11, 2025, from https://docs.langchain.com/oss/python/integrations/providers/overview.

[LangChain] LangChain. Langsmith docs - docs by langchain. Retrieved 2025-11-05, from https://docs.langchain.com/langsmith/home.

[LangGraph] LangGraph. Graph api overview - docs by langchain. Retrieved 2025-11-05, from https://docs.langchain.com/oss/javascript/langgraph/graph-api.

[19] Liu, B., Li, X., Zhang, J., Wang, J., He, T., Hong, S., Liu, H., Zhang, S., Song, K., Zhu, K., Cheng, Y., Wang, S., Wang, X., Luo, Y., Jin, H., Zhang, P., Liu, O., Chen, J., Zhang, H., Yu, Z., Shi, H., Li, B., Wu, D., Teng, F., Jia, X., Xu, J., Xiang, J., Lin, Y., Liu, T., Liu, T., Su, Y., Sun, H., Berseth, G., Nie, J., Foster, I., Ward, L., Wu, Q., Gu, Y., Zhuge, M., Liang, X., Tang, X., Wang, H., You, J., Wang, C., Pei, J., Yang, Q., Qi, X., and Wu, C. (2025). Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. Retrieved ., from https://arxiv.org/abs/2504.01990

[20] Liu, Y., Singh, J., Liu, G., Payani, A., and Zheng, L. (2024). Towards hierarchical multi-agent workflows for zero-shot prompt optimization. Retrieved ., from https://arxiv.org/abs/2405.20252v2

[21] Masterman, T., Besen, S., Sawtell, M., and Chao, A. (2024). The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. Retrieved ., from https://arxiv.org/abs/2404.11584

[22] Niu, B., Song, Y., Lian, K., Shen, Y., Yao, Y., Zhang, K., and Liu, T. (2025). Flow: Modularized agentic workflow automation. Retrieved ., from https://arxiv.org/abs/2501.07834v2

[23] OpenAI (2025). Introducing gpt-5. Retrieved October 29, 2025, from https://openai.com/index/introducing-gpt-5/.

[24] Rettenmeyer, J. (2025). What is cognitive architecture? how intelligent agents think, learn, and adapt. Retrieved October 20, 2025, from https://quiq.com/blog/what-is-cognitive-architecture/.

[25] Sahdev, S. (2025). Understanding ai agent perception: The gateway to smarter, more adaptive systems | article by aryaxai. Retrieved October 22, 2025, from https://www.aryaxai.com/article/understanding-ai-agent-perception-the-gateway-to-smarter-more-adaptive-systems.

[26] Shi, Z., Gao, S., Yan, L., Inc., B., Feng, Y., Chen, X., Chen, Z., Yin, D., Verberne, S., and Ren, Z. (2025). Tool learning in the wild: Empowering language models as automatic tool agents. *Proceedings of the ACM Web Conference 2025 (WWW '25)*, page 17. Retrieved ., from https://arxiv.org/pdf/2405.16533

[27] Shih, Y.-K. and Kang, Y.-K. (2025). Design and implementation of a secure rag-enhanced ai chatbot for smart tourism customer service: Defending against prompt injection attacks – a case study of hsinchu, taiwan. Retrieved ., from https://arxiv.org/abs/2509.21367

[28] Software, S. (2025). Ai agent architecture: Tutorial & examples - fme by safe software. Retrieved October 25, 2025, from https://fme.safe.com/guides/ai-agent-architecture/.

[29] Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. (2024). Cognitive Architectures for Language Agents. Retrieved ., from https://arxiv.org/abs/2309.02427v3

[30] Team, L. (2025a). Multi-agent architecture – how intelligent systems work together. Retrieved October 29, 2025, from https://www.lyzr.ai/blog/multi-agent-architecture/.

[31] Team, S. (2025b). A practical guide to the architectures of agentic applications | speakeasy. Retrieved October 25, 2025, from https://www.speakeasy.com/mcp/using-mcp/ai-agents/architecture-patterns.

[32] Tiwary, S. (2025). Build and manage multi-system agents with vertex ai. Retrieved October 27, 2025, from https://cloud.google.com/blog/products/ai-machine-learning/build-and-manage-multi-system-agents-with-vertex-ai?hl=en.

[33] Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., and Nguyen, H. D. (2025). Multi-agent collaboration mechanisms: A survey of llms. Retrieved ., from https://arxiv.org/abs/2501.06322v1

[34] Tuychiev, B. (2025). Comprehensive guide to building ai agents using google agent development kit (adk). Retrieved October 29, 2025, from https://www.firecrawl.dev/blog/google-adk-multi-agent-tutorial.

[35] Wilsdon, T. (2025). Comprehensive google workspace / g suite mcp server. Access on 2025-11-05 from https://github.com/taylorwilsdon/google_workspace_mcp.

[36] Xeol-Io (2024). GitHub - xeol-io/bumpgen: bumpgen is an AI agent that upgrades npm packages. Retrieved October 16, 2025, from https://github.com/xeol-io/bumpgen.

[37] Zhang, H., Liu, X., Lv, B., Sun, X., Jing, B., Iong, I. L., Hou, Z., Qi, Z., Lai, H., Xu, Y., Lu, R., Wang, H., Tang, J., and Dong, Y. (2025). Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. Retrieved ., from https://www.semanticscholar.org/reader/c4060d004efb85967d6c3f3fd38de437589ce2af